Problem 1:

This is a knn classifier for the the magic gamma telescope dataset (gamma or hadrons)

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,
         f1_score, confusion_matrix, classification_report
```

(1) Load all data

load dataset from UCI repo, features from f1 to f10 and class to represent type gamma or hadron

```
def read_magic_dataset() -> pd.DataFrame:
    column_names =
         ["f1","f2","f3","f4","f5","f6","f7","f8","f9","f10","class",]
    url = ("https://archive.ics.uci.edu/ml/machine-learning-
         databases/magic/magic04.data")
    df = pd.read_csv(url, names=column_names)
    return df
df = read_magic_dataset()
df.head()
```

	f1	f2	f3	f4	f5	f6	f7	f8
0	28.7967	16.0021	2.6449	0.3918	0.1982	27.7004	22.0110	-8.2027
1	31.6036	11.7235	2.5185	0.5303	0.3773	26.2722	23.8238	-9.9574
2	162.0520	136.0310	4.0612	0.0374	0.0187	116.7410	-64.8580	-45.2160
3	23.8172	9.5728	2.3385	0.6147	0.3922	27.2107	-6.4633	-7.1513
4	75.1362	30.9205	3.1611	0.3168	0.1832	-5.5277	28.5525	21.8393

(2) Get balanced dataset :

to prevent bias, both gamma and hadron count should be the same, so the larger one is downsampled to the smaller one

```
def balance_dataset(df: pd.DataFrame) -> pd.DataFrame:
    gamma = df[df["class"] == "g"]
    hadron = df[df["class"] == "h"]
    n = min(len(gamma), len(hadron))
```

```
    np.random.seed(42)
    gamma_balanced = gamma.sample(n=n)
    hadron_balanced = hadron.sample(n=n)
    balanced = pd.concat([gamma_balanced,
         hadron_balanced]).sample(frac=1).reset_index(drop=True)
    return balanced
df = balance_dataset(df)
print(df["class"].value_counts())
```

```
class
h    6688
g    6688
Name: count, dtype: int64
```

(3) Split dataset:

```
 70% -> training set (knn)
 15% -> validation set (best k)
 15% -> test set (final score)
```

```
def split_data(df: pd.DataFrame, training_fractor: float = 0.7,
         validation_fractor: float = 0.15, testing_fractor: float = 0.15,):
    assert abs(training_fractor + validation_fractor + testing_fractor - 1.0)
         < 1e-6
    X = df.drop(columns=["class"]).values
    y = (df["class"] == "g").astype(int).values
    X_training, X_temp, y_training, y_temp = train_test_split(X, y,
         train_size=training_fractor, stratify=y)
    X_validation, X_testing, y_validation, y_testing =
         train_test_split(X_temp, y_temp, train_size=validation_fractor /
         (validation_fractor + testing_fractor), stratify=y_temp)
    return X_training, X_validation, X_testing, y_training, y_validation,
         y_testing
X_training, X_validation, X_testing, y_training, y_validation, y_testing =
         split_data(df)
print(f"Training: {len(X_training)}, Validation: {len(X_validation)},
         Testing: {len(X_testing)}")
```

```
Training: 9363, Validation: 2006, Testing: 2007
```

(4) Evalute knn :

knn classifier for every k and calculate accuracy, precision, recall, f1-score, and confusion matrix

```
def evaluate_knn(X_training, y_training, X_eval, y_eval, ks: list):
    results = {}
    for k in ks:
        classifier = KNeighborsClassifier(n_neighbors=k)
        classifier.fit(X_training, y_training)
        y_pred = classifier.predict(X_eval)
        metrics = {
```

```
            "accuracy": accuracy_score(y_eval, y_pred),
            "precision": precision_score(y_eval, y_pred),
            "recall": recall_score(y_eval, y_pred),
            "f1": f1_score(y_eval, y_pred),
            "confusion_matrix": confusion_matrix(y_eval, y_pred),
            "classification_report": classification_report(y_eval, y_pred,
         output_dict=True),
        }
        results[k] = metrics
        print(f"k = {k}")
        print(f"  accuracy  : {metrics['accuracy']:.4f}")
        print(f"  precision : {metrics['precision']:.4f}")
        print(f"  recall    : {metrics['recall']:.4f}")
        print(f"  f1-score  : {metrics['f1']:.4f}")
        print("  confusion matrix:\n", metrics["confusion_matrix"])
        print("\n")
    return results
```

(5) Best k :

k values from 1 to 20 are used to find the best k on the validation set and the k with the highest f1-score is chosen

```
ks = list(range(1, 21))
print("Evaluating on validation set")
val_results = evaluate_knn(X_training, y_training, X_validation,
         y_validation, ks)

best_k = max(val_results, key=lambda k: val_results[k]["f1"])
print(f"Best k: {best_k}")
```

```
Evaluating on validation set
k = 1
  accuracy  : 0.7368
  precision : 0.7165
  recall    : 0.7836
  f1-score  : 0.7486
  confusion matrix:
 [[692 311]
 [217 786]]


k = 2
  accuracy  : 0.7338
  precision : 0.7836
  recall    : 0.6461
  f1-score  : 0.7082
  confusion matrix:
 [[824 179]
 [355 648]]
```

```
k = 3
  accuracy  : 0.7502
  precision : 0.7168
  recall    : 0.8275
  f1-score  : 0.7682
  confusion matrix:
 [[675 328]
 [173 830]]


k = 4
  accuracy  : 0.7592
  precision : 0.7610
  recall    : 0.7557
  f1-score  : 0.7584
  confusion matrix:
 [[765 238]
 [245 758]]


k = 5
  accuracy  : 0.7562
  precision : 0.7189
  recall    : 0.8415
  f1-score  : 0.7754
  confusion matrix:
 [[673 330]
 [159 844]]


k = 6
  accuracy  : 0.7662
  precision : 0.7538
  recall    : 0.7906
  f1-score  : 0.7718
  confusion matrix:
 [[744 259]
 [210 793]]


k = 7
  accuracy  : 0.7612
  precision : 0.7183
  recall    : 0.8594
  f1-score  : 0.7826
  confusion matrix:
 [[665 338]
 [141 862]]


k = 8
  accuracy  : 0.7617
  precision : 0.7397
  recall    : 0.8076
```

```
  f1-score  : 0.7722
  confusion matrix:
 [[718 285]
 [193 810]]


k = 9
  accuracy  : 0.7577
  precision : 0.7149
  recall    : 0.8574
  f1-score  : 0.7797
  confusion matrix:
 [[660 343]
 [143 860]]


k = 10
  accuracy  : 0.7617
  precision : 0.7346
  recall    : 0.8195
  f1-score  : 0.7747
  confusion matrix:
 [[706 297]
 [181 822]]


k = 11
  accuracy  : 0.7582
  precision : 0.7166
  recall    : 0.8544
  f1-score  : 0.7794
  confusion matrix:
 [[664 339]
 [146 857]]


k = 12
  accuracy  : 0.7557
  precision : 0.7236
  recall    : 0.8275
  f1-score  : 0.7721
  confusion matrix:
 [[686 317]
 [173 830]]


k = 13
  accuracy  : 0.7577
  precision : 0.7142
  recall    : 0.8594
  f1-score  : 0.7801
  confusion matrix:
 [[658 345]
 [141 862]]
```

```
k = 14
  accuracy  : 0.7592
  precision : 0.7253
  recall    : 0.8345
  f1-score  : 0.7761
  confusion matrix:
 [[686 317]
 [166 837]]


k = 15
  accuracy  : 0.7562
  precision : 0.7110
  recall    : 0.8634
  f1-score  : 0.7798
  confusion matrix:
 [[651 352]
 [137 866]]


k = 16
  accuracy  : 0.7627
  precision : 0.7254
  recall    : 0.8455
  f1-score  : 0.7808
  confusion matrix:
 [[682 321]
 [155 848]]


k = 17
  accuracy  : 0.7562
  precision : 0.7113
  recall    : 0.8624
  f1-score  : 0.7796
  confusion matrix:
 [[652 351]
 [138 865]]


k = 18
  accuracy  : 0.7597
  precision : 0.7232
  recall    : 0.8415
  f1-score  : 0.7779
  confusion matrix:
 [[680 323]
 [159 844]]


k = 19
  accuracy  : 0.7577
```

```
  precision : 0.7131
  recall    : 0.8624
  f1-score  : 0.7807
  confusion matrix:
 [[655 348]
 [138 865]]


k = 20
  accuracy  : 0.7572
  precision : 0.7183
  recall    : 0.8465
  f1-score  : 0.7771
  confusion matrix:
 [[670 333]
 [154 849]]


Best k: 7
```

(6) Evaluation :

evaluate model on test set using the best k chosen

```
print("Evaluating best model on test set")
test_results = evaluate_knn(X_training, y_training, X_testing, y_testing,
         [best_k])

report = test_results[best_k]["classification_report"]
print(f"Class 0 (Gamma)  - Precision: {report['0']['precision']:.4f}, Recall:
         {report['0']['recall']:.4f}, F1: {report['0']['f1-score']:.4f}")
print(f"Class 1 (Hadron) - Precision: {report['1']['precision']:.4f}, Recall:
         {report['1']['recall']:.4f}, F1: {report['1']['f1-score']:.4f}")
print(f"Overall Accuracy : {report['accuracy']:.4f}")
```

```
Evaluating best model on test set
k = 7
  accuracy  : 0.7598
  precision : 0.7202
  recall    : 0.8495
  f1-score  : 0.7795
  confusion matrix:
 [[673 331]
 [151 852]]


Class 0 (Gamma)  - Precision: 0.8167, Recall: 0.6703, F1: 0.7363
Class 1 (Hadron) - Precision: 0.7202, Recall: 0.8495, F1: 0.7795
Overall Accuracy : 0.7598
```